DIOWKRAD SF2 CHIP-TUNE ENGINE — Спецификация (v1.2.5 Final)

О проекте

Финальная сборка движка для синтеза звука с эмуляцией чипа 2A03 (NES), поддержкой 4 типов волн (синус, пила, квадрат, треугольник), MIDI на 16 каналов, роллом и кастомной клавиатурой.
Без внешних библиотек, без дебага, без changelog. Полностью оффлайн.

- **Архитектурные решения (зафиксированы как финальные):**
  - Переключатель октав и C4 по умолчанию — финальное решение (меньше багов, выглядит законченно).
  - Дебаг-код исключён из релиза.
  - Changelog-панель не включается в финальный релиз.
  - Все стили — через CSS-переменные; JS не меняет стили напрямую.
  - Проект не должен зависеть от интернета.

Порядок загрузки скриптов (критично)

Скрипты должны подключаться в этом порядке в `index.html` (через `<script>` в конце `<body>`). **Менять порядок нельзя — нарушать зависимости.**

1. `core-init.js` — инициализация ядра, проверка DOM, установка констант.
2. `synth-engine.js` — ядро синтезатора, эмуляция 2A03.
3. `wave-generator.js` — генерация 4 волн.
4. `midi-handler.js` — обработка MIDI, захват каналов 1–16.
5. `midi-select.js` — построение селектора голосов.
6. `keyboard-render.js` — отрисовка клавиш, полифония.
7. `octave-control.js` — переключатель октав, границы.
8. `roll-render.js` — ролл (ноты по времени).
9. `roll-scroll.js` — скролл ролла, синхронизация.
10. `ui-controls.js` — группировка UI: переключатели, чекбоксы, скролы.

Если добавить новый модуль — ставь его в конец и явно пропиши зависимости.

Карта модулей (с привязкой к реальным функциям)

Ключевые ID и их назначение (чтобы не сломать при правке)

Управление

- `start-btn`: кнопка запуска синтезатора → вызывает `SynthAPI.init()`.

Модуль	Назначение	Ключевые функции / логика	Ключевые ID / селекторы	Зависимости
`core-init.js`	Инициализация ядра, проверка DOM, запуск SynthAPI, установка границ нот	`SynthAPI.init()`, `PC_MIN()`, `PC_MAX()`, проверка наличия элементов	`start-btn`, `mute-btn`	—
`synth-engine.js`	Ядро синтезатора, эмуляция 2A03, логика звука	Эмуляция регистров чипа, распределение по волнам, обработка полифонии	Внутренние переменные движка	`core-init.js`
`wave-generator.js`	Генерация 4 типов волн: синус, пила, квадрат, треугольник	Выбор типа волны, нормализация амплитуды, подготовка формы волны	`.wave-type`	`synth-engine.js`
`midi-handler.js`	Обработка MIDI, захват каналов 1–16, распределение сигнала	Приём MIDI-сообщений, маппинг на каналы, распределение на 16 MIDI-каналов, поддержка нескольких MIDI-устройств	`.midi-status`, `#midiVoiceSel`	`synth-engine.js`

midi-select.js	Построение селектора голосов (опгрупп по диапазонам)	buildMidiSel(): создание optgroup, заполнение селектора	#midiVoiceSel	midi-handler.js
keyboard-render.js	Отрисовка клавиш, логика полифонии, обработка ввода	buildKeys(), логика нажатия, обработка полифонического ввода	.keyboard, .key	core-init.js, synth-engine.js
octave-control.js	Переключатель октав, установка границ, отображение текущей октавы	Логика переключения, установка currentOctave, обновление отображения	.octave-controls, #octave-display	keyboard-render.js
roll-render.js	Ролл (ноты по времени), отрисовка нот, синхронизация с клавиатурой	Отрисовка .roll-note, привязка к временной шкале, синхронизация высоты ноты с клавиатурой	#roll-container, .roll-note	keyboard-render.js, octave-control.js
roll-scroll.js	Скролл ролла, горизонтальная прокрутка, синхронизация скролла с временем	Управление #roll-viewport, синхронизация скролла и временной позиции	#roll-viewport, .roll-track	roll-render.js
ui-controls.js	Группировка UI: переключатели отдельно, чекбоксы и скролы отдельно	Логика группировки, управление видимостью групп, обработка событий UI	.ui-group-switches, .ui-group-checks	—

mute-btn: кнопка глушения звука → переключает глобальное отключение аудио.

MIDI

- `#midiVoiceSel`: селектор голосов → заполняется функцией `buildMidiSel()` из `midi-select.js`.
- `.midi-status`: индикатор статуса MIDI (подключено/ошибка/активно).

Клавиатура

- `.keyboard`: контейнер всех клавиш.
- `.key`: отдельные клавиши; каждая имеет атрибуты, связанные с нотой и октавой.

Октавы

- `.octave-controls`: группа переключателей октав.
- `#octave-display`: текстовое отображение текущей октавы (например, «C4»).

Ролл

- `#roll-container`: общий контейнер ролла.
- `#roll-viewport`: область прокрутки (overflow: hidden, scrollable).
- `.roll-note`: отдельная нота в ролле.
- `.roll-track`: дорожка/временная шкала (фон, разметка тактов).

UI-группы (по твоей задумке — сгруппировано отдельно)

- `.ui-group-switches`: группа переключателей (например, режимы, типы волн).
- `.ui-group-checks`: группа чекбоксов и ползунков (полифония, эффекты и т.п.).

Важные константы и границы (единый источник истины)

Все границы и константы хранятся в `core-init.js`. **Не дублируйте их в других модулях.**

- `PC_MIN()` = 24 — минимальная нота (C1).
- `PC_MAX()` = 78 — максимальная нота (C8).
  Это единый диапазон для обоих режимов (клавиатура и ролл).
- `currentOctave` = 4 — начальная октава по умолчанию (C4).

  При изменении границ — меняйте только в `core-init.js`. Все остальные модули должны брать значения оттуда.

Привязка к твоим реальным функциям (чтобы сразу было понятно, где искать)

- `buildMidiSel()` — находится в `midi-select.js`, заполняет `#midiVoiceSel`. Не вызывай вручную из других модулей — он вызывается автоматически при инициализации MIDI.
- `buildKeys()` — находится в `keyboard-render.js`. Перерисовывает клавиши при смене октавы или границ. Не меняй логику внутри, если не понимаешь, как это влияет на полифонию.
- Границы нот `PC_MIN` и `PC_MAX` — используются в `buildKeys()` и в отрисовке ролла. Если их поменять, нужно проверить, чтобы ролл не «уехал» по высоте.
- Логика распределения MIDI по 16 каналам — в `midi-handler.js`. Не меняй эту логику без обновления спецификации: она привязана к эмуляции 2A03 и может сломать звук.

Архитектурные нюансы (почему сделано так, а не иначе)

- **Полифония и ввод:** Полифонический ввод обрабатывается в `keyboard-render.js`, а не в MIDI-модуле. Это сделано, чтобы не смешивать логику MIDI и логику клавиатуры.
- **Ролл и скролл:** Скролл ролла (`roll-scroll.js`) не управляет звуком, только отображением. Синхронизация с временем — через отдельный механизм, который не ломает аудио-поток.
- **Группировка UI:** Переключатели, чекбоксы и скролы вынесены в отдельные группы, чтобы упростить верстку и не ломать макет при добавлении новых элементов.
- **Эмуляция 2A03:** Логика эмуляции вынесена в `synth-engine.js`, чтобы не засорять другие модули низкоуровневыми деталями.

Рекомендации для внесения изменений

1. **Перед правкой модуля — проверь зависимости в таблице.** Если модуль зависит от другого, убедись, что он загружается раньше.
2. **Не меняй ID без обновления спецификации.** Если нужно переименовать ID, обнови таблицу и все упоминания в коде.
3. **Новые модули добавляй в конец списка**, явно указывая зависимости.
4. **При изменении границ нот — меняй только в `core-init.js`.** Проверь, что это не ломает отрисовку клавиатуры и ролла.
5. **Если добавляешь новую функцию — опиши её в спецификации.** Это поможет быстрее разобраться в будущем.

Статус

- Версия: v1.2.5 «Final».
- Разработка завершена. Дальнейшее расширение не планируется.
- Спецификация актуальна на момент сборки релиза.